F O O D R U S H

Database Documentation

Food Delivery Management & Analytics System

SQL Project

MySQL · MySQL Workbench · Git / GitHub

Table of Contents

- 1. Introduction & Scope
- 2. System Modules
- 3. Entity Overview
- 4. Entity-Relationship Diagram (Textual)
- 5. Relationship Summary
- 6. Detailed Table Design (all 12 tables)
- 7. Keys & Referential Integrity Summary
- 8. Normalization (1NF, 2NF, 3NF)
- 9. Constraints Summary
- 10. Advanced Database Objects (Views, Procedures, Functions, Triggers, Events)
- 11. Design Decisions & Rationale
- 12. Conclusion

1. Introduction & Scope

FoodRush is a food delivery platform connecting customers, restaurants, delivery partners, and food items. This document defines the complete database design for FoodRush: every entity, its attributes, keys, constraints, and the relationships that connect them, along with the normalization reasoning behind the design and the advanced database objects (views, procedures, functions, triggers, events) built on top of the schema.

The design goal is a normalized relational schema (up to 3NF) that accurately reflects real-world food delivery operations — supporting customer management, restaurant and menu management, order processing, payments, delivery tracking, coupon management, and review/rating management — while remaining efficient to query for business analytics.

2. System Modules

- Customer Management
- Restaurant Management
- Menu Management
- Order Management
- Payment Management
- Delivery Management
- Coupon Management
- Review & Rating Management

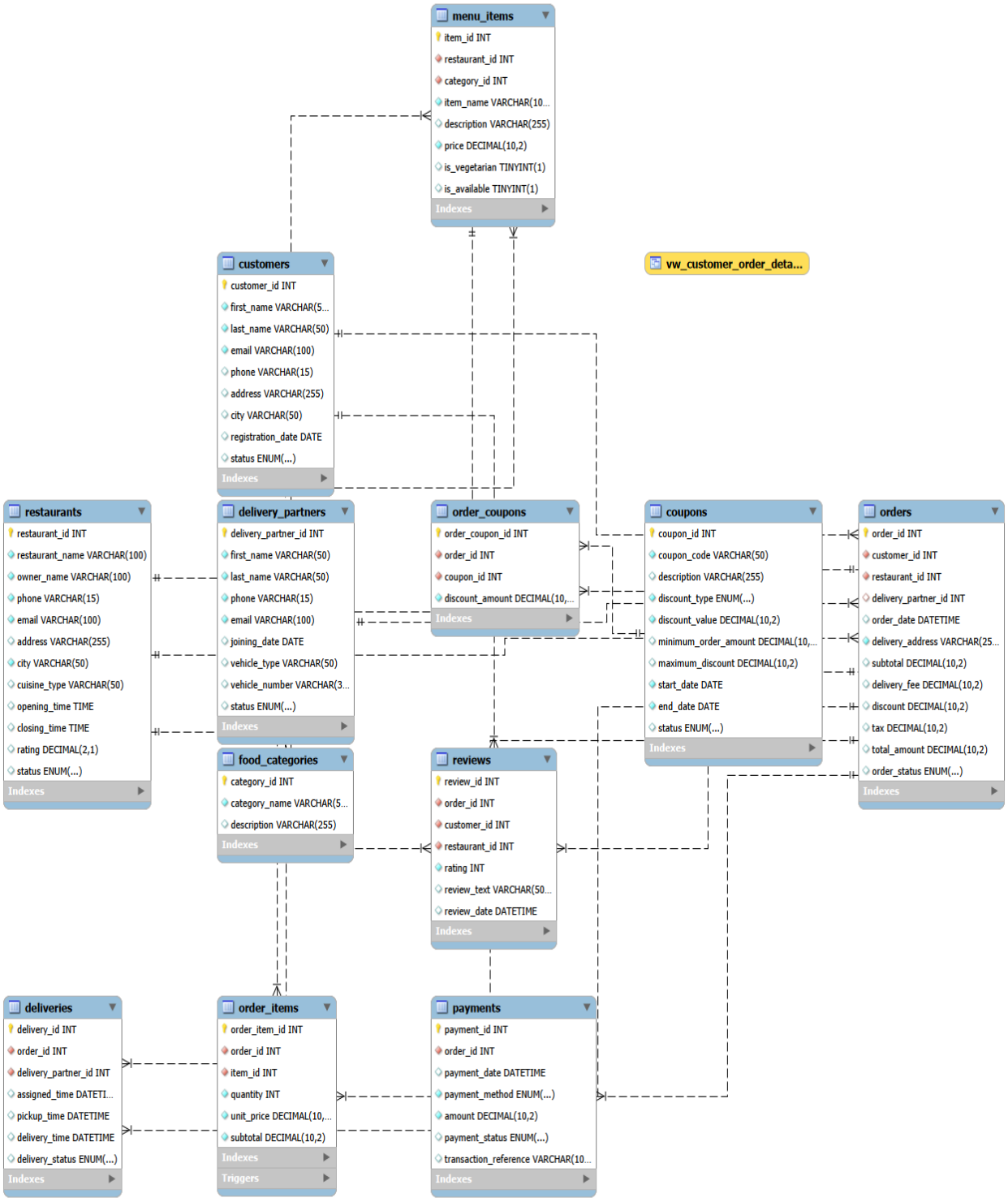
3. Entity Overview

The schema consists of 12 tables, grouped by the module they primarily serve:

#	Table	Module	Purpose (one line)
1	Customers	Customer Mgmt	Registered platform users who place orders
2	Restaurants	Restaurant Mgmt	Partner restaurants listed on the platform
3	Food_Categories	Menu Mgmt	Lookup table of food categories
4	Menu_Items	Menu Mgmt	Food items offered by each restaurant
5	Delivery_Partners	Delivery Mgmt	Riders who fulfill deliveries
6	Orders	Order Mgmt	Core transactional order record
7	Order_Items	Order Mgmt	Line items belonging to an order
8	Payments	Payment Mgmt	Payment transactions against orders
9	Deliveries	Delivery Mgmt	Delivery lifecycle tracking per order
10	Coupons	Coupon Mgmt	Discount coupon definitions
11	Order_Coupons	Coupon Mgmt	Coupons applied to specific orders
12	Reviews	Review Mgmt	Customer ratings/feedback per order

4. Entity-Relationship Diagram (Textual)

The diagram below represents the relationship structure of the schema



5. Relationship Summary

Parent Table	Child Table	Type	Foreign Key (in child)
Customers	Orders	1 : N	Orders.customer_id
Restaurants	Orders	1 : N	Orders.restaurant_id
Delivery_Partners	Orders	1 : N (nullable)	Orders.delivery_partner_id
Restaurants	Menu_Items	1 : N	Menu_Items.restaurant_id
Food_Categories	Menu_Items	1 : N	Menu_Items.category_id
Orders	Order_Items	1 : N	Order_Items.order_id
Menu_Items	Order_Items	1 : N	Order_Items.item_id
Orders	Payments	1 : N*	Payments.order_id
Orders	Deliveries	1 : 1	Deliveries.order_id (UNIQUE)
Delivery_Partners	Deliveries	1 : N	Deliveries.delivery_partner_id
Orders	Order_Coupons	1 : N	Order_Coupons.order_id
Coupons	Order_Coupons	1 : N	Order_Coupons.coupon_id
Orders	Reviews	1 : N	Reviews.order_id
Customers	Reviews	1 : N	Reviews.customer_id
Restaurants	Reviews	1 : N	Reviews.restaurant_id

6. Detailed Table Design

6.1 Customers

Stores registered customer details.

Column	Data Type	Key	Constraints / Notes
customer_id	INT	PK	AUTO_INCREMENT
first_name	VARCHAR(50)		NOT NULL
last_name	VARCHAR(50)		NOT NULL
email	VARCHAR(100)		NOT NULL, UNIQUE
phone	VARCHAR(15)		NOT NULL, UNIQUE
address	VARCHAR(255)		
city	VARCHAR(50)		NOT NULL
registration_date	DATE		NOT NULL, DEFAULT CURRENT_DATE
status	ENUM		'Active','Inactive','Blocked'; DEFAULT 'Active'

6.2 Restaurants

Stores partner restaurant profile details.

Column	Data Type	Key	Constraints / Notes
restaurant_id	INT	PK	AUTO_INCREMENT
restaurant_name	VARCHAR(100)		NOT NULL
owner_name	VARCHAR(100)		NOT NULL
phone	VARCHAR(15)		NOT NULL, UNIQUE
email	VARCHAR(100)		UNIQUE
address	VARCHAR(255)		NOT NULL
city	VARCHAR(50)		NOT NULL
cuisine_type	VARCHAR(50)		
opening_time	TIME		NOT NULL
closing_time	TIME		NOT NULL
rating	DECIMAL(2,1)		CHECK (0–5), DEFAULT 0.0
status	ENUM		'Active','Inactive','Closed'; DEFAULT 'Active'

6.3 Food_Categories

Lookup table of food categories used to classify menu items.

Column	Data Type	Key	Constraints / Notes
category_id	INT	PK	AUTO_INCREMENT
category_name	VARCHAR(50)		NOT NULL, UNIQUE
description	VARCHAR(255)		

6.4 Menu_Items

Food items offered by a specific restaurant.

Column	Data Type	Key	Constraints / Notes
item_id	INT	PK	AUTO_INCREMENT
restaurant_id	INT	FK	→ Restaurants.restaurant_id, NOT NULL, ON DELETE CASCADE
category_id	INT	FK	→ Food_Categories.category_id, NOT NULL, ON DELETE RESTRICT
item_name	VARCHAR(100)		NOT NULL
description	VARCHAR(255)		
price	DECIMAL(8,2)		NOT NULL, CHECK (price > 0)
is_vegetarian	BOOLEAN		NOT NULL, DEFAULT TRUE
is_available	BOOLEAN		NOT NULL, DEFAULT TRUE

6.5 Delivery_Partners

Delivery staff details and current availability.

Column	Data Type	Key	Constraints / Notes
delivery_partner_id	INT	PK	AUTO_INCREMENT
first_name	VARCHAR(50)		NOT NULL
last_name	VARCHAR(50)		NOT NULL
phone	VARCHAR(15)		NOT NULL, UNIQUE
email	VARCHAR(100)		UNIQUE
joining_date	DATE		NOT NULL, DEFAULT CURRENT_DATE
vehicle_type	ENUM		'Bike','Scooter','Bicycle','Car'
vehicle_number	VARCHAR(20)		

6.6 Orders

The central transactional table — one row per customer order.

Column	Data Type	Key	Constraints / Notes
order_id	INT	PK	AUTO_INCREMENT
customer_id	INT	FK	→ Customers.customer_id, NOT NULL, ON DELETE RESTRICT
restaurant_id	INT	FK	→ Restaurants.restaurant_id, NOT NULL, ON DELETE RESTRICT
delivery_partner_id	INT	FK	→ Delivery_Partners.delivery_partner_id, NULLABLE, ON DELETE SET NULL
order_date	DATETIME		NOT NULL, DEFAULT CURRENT_TIMESTAMP
delivery_address	VARCHAR(255)		NOT NULL
subtotal	DECIMAL(10,2)		NOT NULL, DEFAULT 0.00
delivery_fee	DECIMAL(8,2)		NOT NULL, DEFAULT 0.00
discount	DECIMAL(8,2)		NOT NULL, DEFAULT 0.00
tax	DECIMAL(8,2)		NOT NULL, DEFAULT 0.00
total_amount	DECIMAL(10,2)		NOT NULL, DEFAULT 0.00
order_status	ENUM		'Placed','Confirmed','Preparing','Out for Delivery','Delivered','Cancelled','Rejected'

6.7 Order_Items

Bridge table resolving the many-to-many relationship between Orders and Menu_Items.

Column	Data Type	Key	Constraints / Notes
order_item_id	INT	PK	AUTO_INCREMENT
order_id	INT	FK	→ Orders.order_id, NOT NULL, ON DELETE CASCADE
item_id	INT	FK	→ Menu_Items.item_id, NOT NULL, ON DELETE RESTRICT
quantity	INT		NOT NULL, CHECK (quantity > 0)
unit_price	DECIMAL(8,2)		NOT NULL
subtotal	DECIMAL(10,2)		NOT NULL

6.8 Payments

Payment transaction record for an order.

Column	Data Type	Key	Constraints / Notes
payment_id	INT	PK	AUTO_INCREMENT
order_id	INT	FK	→ Orders.order_id, NOT NULL, ON DELETE CASCADE
payment_date	DATETIME		NOT NULL, DEFAULT CURRENT_TIMESTAMP
payment_method	ENUM		'UPI','Credit Card','Debit Card','Cash','Wallet','Net Banking'
amount	DECIMAL(10,2)		NOT NULL
payment_status	ENUM		'Successful','Pending','Failed','Refunded'; DEFAULT 'Pending'
transaction_reference	VARCHAR(100)		UNIQUE

6.9 Deliveries

Tracks the delivery lifecycle for each order.

Column	Data Type	Key	Constraints / Notes
delivery_id	INT	PK	AUTO_INCREMENT
order_id	INT	FK, UNIQUE	→ Orders.order_id, NOT NULL, ON DELETE CASCADE
delivery_partner_id	INT	FK	→ Delivery_Partners.delivery_partner_id, NULLABLE, ON DELETE SET NULL
assigned_time	DATETIME		
pickup_time	DATETIME		
delivery_time	DATETIME		
delivery_status	ENUM		'Assigned','Picked Up','Delivered','Failed','Cancelled'

6.10 Coupons

Discount coupon definitions, independent of any single order.

Column	Data Type	Key	Constraints / Notes
coupon_id	INT	PK	AUTO_INCREMENT
coupon_code	VARCHAR(30)		NOT NULL, UNIQUE
description	VARCHAR(255)		
discount_type	ENUM		'Percentage','Flat'

Column	Data Type	Key	Constraints / Notes
discount_value	DECIMAL(8,2)		NOT NULL, CHECK (discount_value > 0)
minimum_order_amount	DECIMAL(8,2)		DEFAULT 0.00
maximum_discount	DECIMAL(8,2)		
start_date	DATE		NOT NULL
end_date	DATE		NOT NULL, CHECK (end_date >= start_date)
status	ENUM		'Active','Inactive','Expired'; DEFAULT 'Active'

6.11 Order_Coupons

Bridge table resolving the many-to-many relationship between Orders and Coupons.

Column	Data Type	Key	Constraints / Notes
order_coupon_id	INT	PK	AUTO_INCREMENT
order_id	INT	FK	→ Orders.order_id, NOT NULL, ON DELETE CASCADE
coupon_id	INT	FK	→ Coupons.coupon_id, NOT NULL, ON DELETE RESTRICT
discount_amount	DECIMAL(8,2)		NOT NULL

6.12 Reviews

Customer ratings and feedback tied to a specific order.

Column	Data Type	Key	Constraints / Notes
review_id	INT	PK	AUTO_INCREMENT
order_id	INT	FK	→ Orders.order_id, NOT NULL, ON DELETE CASCADE
customer_id	INT	FK	→ Customers.customer_id, NOT NULL, ON DELETE CASCADE
restaurant_id	INT	FK	→ Restaurants.restaurant_id, NOT NULL, ON DELETE CASCADE
rating	TINYINT		NOT NULL, CHECK (rating BETWEEN 1 AND 5)
review_text	VARCHAR(500)		
review_date	DATETIME		NOT NULL, DEFAULT CURRENT_TIMESTAMP

7. Keys & Referential Integrity Summary

Primary Keys

Every table uses a single-column surrogate primary key (an auto-incrementing integer ID) rather than a natural/composite key. This keeps joins simple and fast, and insulates the schema from changes to naturally-unique-seeming values (e.g., email or phone) which could theoretically change or be reused.

Foreign Key Delete Rules

Rule	Used On	Reasoning
CASCADE	Menu_Items→Restaurants, Order_Items→Orders, Payments→Orders, Deliveries→Orders, Order_Coupons→Orders, Reviews→Orders/Customers/Restaurants	If the parent (e.g., an Order) is deleted, its dependent child records (items, payment, delivery, review) no longer make sense on their own and should be removed together.
RESTRICT	Orders→Customers, Orders→Restaurants, Order_Items→Menu_Items, Order_Coupons→Coupons	Prevents deleting a Customer/Restaurant/Menu Item/Coupon that still has historical orders referencing it — protects transactional history from being silently destroyed.
SET NULL	Orders→Delivery_Partners, Deliveries→Delivery_Partners	If a delivery partner record is removed (e.g., they leave the company), past orders/deliveries should remain, just without a valid partner reference, rather than being deleted or blocked.

8. Normalization

1NF — First Normal Form

Requirement: atomic values, no repeating groups, each column holds a single value.

Applied: Food items within an order are never stored as a comma-separated list inside Orders — they are split into individual rows in Order_Items, one row per item per order. Every column across all 12 tables holds a single atomic value (e.g., no combined 'name' field holding both first and last name).

2NF — Second Normal Form

Requirement: already in 1NF, and every non-key column depends on the whole primary key, not just part of it.

Applied: this rule matters most for tables that conceptually represent a combination of two other entities, like Order_Items. Although Order_Items uses a single surrogate key (order_item_id) rather than a composite key of (order_id, item_id), the non-key columns quantity, unit_price, and subtotal are still logically dependent on the full combination of order and item — not on order_id alone (different items in the same order have different quantities/prices) and not on item_id alone (the same item can appear in many orders with different quantities).

3NF — Third Normal Form

Requirement: already in 2NF, and no transitive dependencies — non-key columns depend only on the primary key, not on other non-key columns.

Applied: Orders does not store restaurant_city, restaurant_name, or customer_name directly — these are transitively dependent on restaurant_id and customer_id respectively, which are already stored as foreign keys. Storing them redundantly in Orders would mean that changing a restaurant's city would require updating every historical order row referencing that restaurant, and would risk the columns becoming inconsistent over time. Instead, this data is retrieved via a JOIN when needed

9. Constraints Summary

Constraint Type	Example	Purpose
NOT NULL	Customers.email, Orders.customer_id	Ensures mandatory business data is always captured
UNIQUE	Customers.email, Coupons.coupon_code	Prevents duplicate accounts / duplicate coupon codes
CHECK	Reviews.rating BETWEEN 1 AND 5	Enforces valid value ranges at the database level, independent of application code
DEFAULT	Orders.order_status = 'Placed'	Provides sensible starting values without requiring the application to set every column explicitly
FOREIGN KEY	Orders.customer_id → Customers.customer_id	Enforces referential integrity — an order can never reference a customer that doesn't exist
ENUM	Orders.order_status, Payments.payment_method	Restricts a column to a fixed, known set of valid business states

10. Advanced Database Objects

Views

vw_customer_order_details — Customer + order + restaurant join for quick order lookups.

vw_restaurant_performance — Per-restaurant totals: orders, revenue, avg order value, rating.

vw_delivery_partner_performance — Per-partner totals: deliveries, avg delivery time, avg rating.

Stored Procedures

sp_place_order — Creates an order + order item in one transaction; returns the new order_id via an OUT parameter.

sp_customer_spending_report — Given a customer_id, returns a full spending summary and segment classification.

Functions

fn_calculate_delivery_charge — Tiered distance-based delivery fee calculator.

11. Conclusion

The FoodRush schema is normalized to 3NF, uses surrogate primary keys with consistent foreign-key delete semantics, and separates concerns across 12 focused tables rather than a small number of overloaded ones. Every table, relationship, and constraint in this design maps directly to a real operational need in a food delivery business, rather than existing purely to satisfy a checklist — which is the core design philosophy behind this project.